

.NET UNIVERSE | BUSAN EDITION · 2608

격월 부산×서울 지역 닷넷 개발자 밋업 · 50분 세션

사용할수록 정리되는 바이브 코딩

운영 프롬프트로 Agent가
비정형→정형 · 수동→자동으로 이끄는 방법

80:20 → 20:80

발표 · 디모이



HELLO, I'M

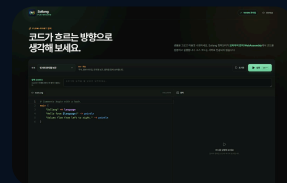
디모이

기술을 가르치고, 커뮤니티를 잇고,
새로운 것을 직접 만듭니다.

01 신구대학교 컴퓨터소프트웨어학과 겸임교수

02 닷넷데브(.NET Dev) 운영진

03 Microsoft MVP



Sollang

복잡한 아이디어를 명확하고 조합 가능한 코드로 표현하기 위해 만든 네이티브 프로그래밍 언어

[GitHub ↗](#)

[Playground ↗](#)

가볍게 시작하겠습니다

AI에게 부탁해서 뭔가 작동하게 만들어본 적 있나요?

문서 정리 · 엑셀 자동화 · 간단한 앱 · 반복 업무

첫 성공의 함정

“앱이 만들어졌습니다.” 그런데 다음 달에도 됩니까?

담당자가 바뀌어도?

설명을 처음부터 다시 하지 않고

입력이 달라져도?

새로운 사례를 안전하게 처리하고

실수해도?

어디가 잘못됐는지 확인할 수 있는가



오늘 끝까지 가져갈 한 질문

첫 번째보다 두 번째 실행이 더 쉬워졌는가?

판정 기준은 프롬프트의 길이가 아니라
재설명 · 수동 판단 · 오류가 줄었는가입니다.

우리가 실제로 받는 입력

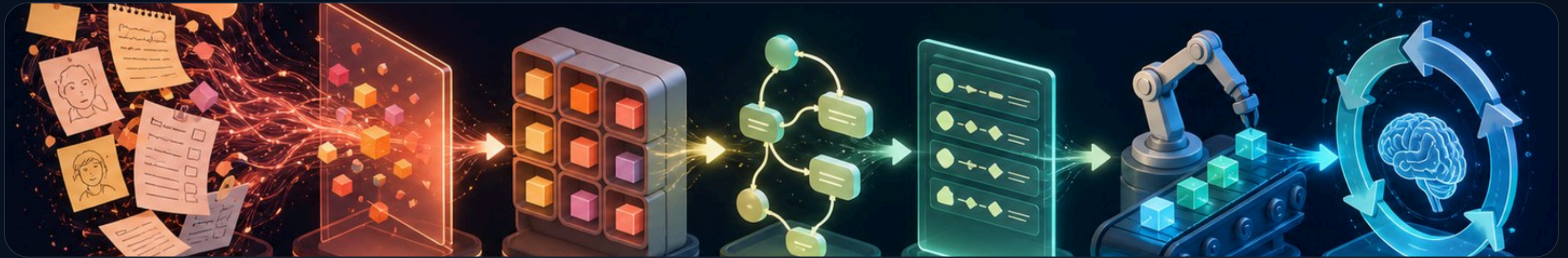
현실의 업무는 깔끔한 표로 오지 않습니다

- 카카오톡 대화와 전화 메모
- 제각각인 신청서와 문서
- “보통은 이렇게 해요”라는 경험
- 상황마다 달라지는 예외

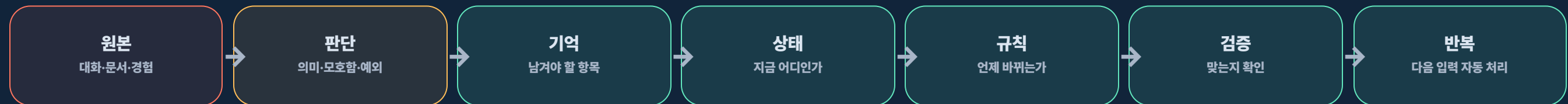
“사람은 맥락으로 이해하지만,
반복하려면 기억할 칸이 필요합니다.”

정형화의 진짜 의미

정형화는 표 한 장이 아니라 판단이 이동하는 과정입니다



비정형에서 정형으로 — 한 번에 점프하지 않는다



결과물과 솔루션의 차이

화면이 있느냐보다 다시 돌릴 수 있느냐

판정 기준	일회성 결과	반복 가능한 솔루션
다음 입력	프롬프트를 다시 설명	정해진 칸에 넣으면 처리
예외	그때그때 수동 수정	규칙으로 흡수
검증	눈으로 대충 확인	체크 기준과 기록이 있음
담당자	만든 사람만 이해	다른 사람도 같은 흐름 사용

근거를 섞지 않겠습니다

이 발표는 세 층으로 말합니다

검증된 사실

AI는 추출·구조화·실행 자동화에
실제 사용된다

1

연구가 지지하는 판단

성과는 지식·업무 구조·검증
환경에 좌우된다

2

발표자의 모델

성숙 방향을
80:20→20:80으로 기억한다

3

1층 · 검증된 사실

AI는 비정형 정보에서 정해진 항목을 추출할 수 있습니다

문서·이미지

필드, 표, 키-값 쌍을 추출

출력 계약

정해진 스키마에 맞춘 구조화 출력

후속 처리

저장·검토·업무 흐름으로 연결

권위 범위: 공급자의 공식 기능·참조 아키텍처를 확인하는 1차 자료

1층 · '알아서'의 실제 의미

운영 프롬프트가 방향을 잡으면 Agent가 다음 단계를 스스로 선택합니다



알아서 = 질문하고, 결과를 보고, 다음 행동을 고르는 순환

2층 · 여러 연구가 지지하는 판단

AI를 잘 쓰는 차이는 현장을 얼마나 아는가에서 커집니다

40만+

Claude Code 상호작용 관찰

숙련자는 계획·검증·맥락 제공을 더 적극적으로 사용

“비개발자의 약점은 코드를 모르는 것이 아니라,
업무를 설명할 언어가 없는 것입니다.”

2층 · 여러 연구가 지지하는 판단

AI는 조직의 강점을 확대하고 약점도 확대합니다

빠른 피드백

결과를 작게 만들고 빨리 확인

명확한 품질 기준

“좋다”를 검사 가능한 조건으로

안전한 변경 흐름

잘못되면 찾고 되돌릴 수 있게

수치를 정직하게 읽기

AI가 항상 빠르다는 결론은 근거와 맞지 않습니다

연구 맥락	관찰 결과	읽어야 할 한계
GitHub 통제 과제	55.8% 더 빠름	좁은 과제·특정 도구
Google 기업 RCT	약 21% 시간 감소 추정	신뢰구간이 넓음
METR 숙련 OSS 개발자	초기 2025 도구에서 19% 느림	16명·익숙한 저장소

공통 결론: 도구 효과는 맥락과 구조에 따라 달라진다.

3층 · 발표자의 모델

비정형 중심에서 정형·반복 중심으로



80:20



해석·탐색

구조·반복

사람이 매번 지시

→ Agent가 다음 단계 주도



20:80



새 판단

재사용 처리

사람이 반복 실행

→ 새 판단만 사람에게

* 연구 수치가 아니라, 성숙 방향을 기억하기 위한 상징

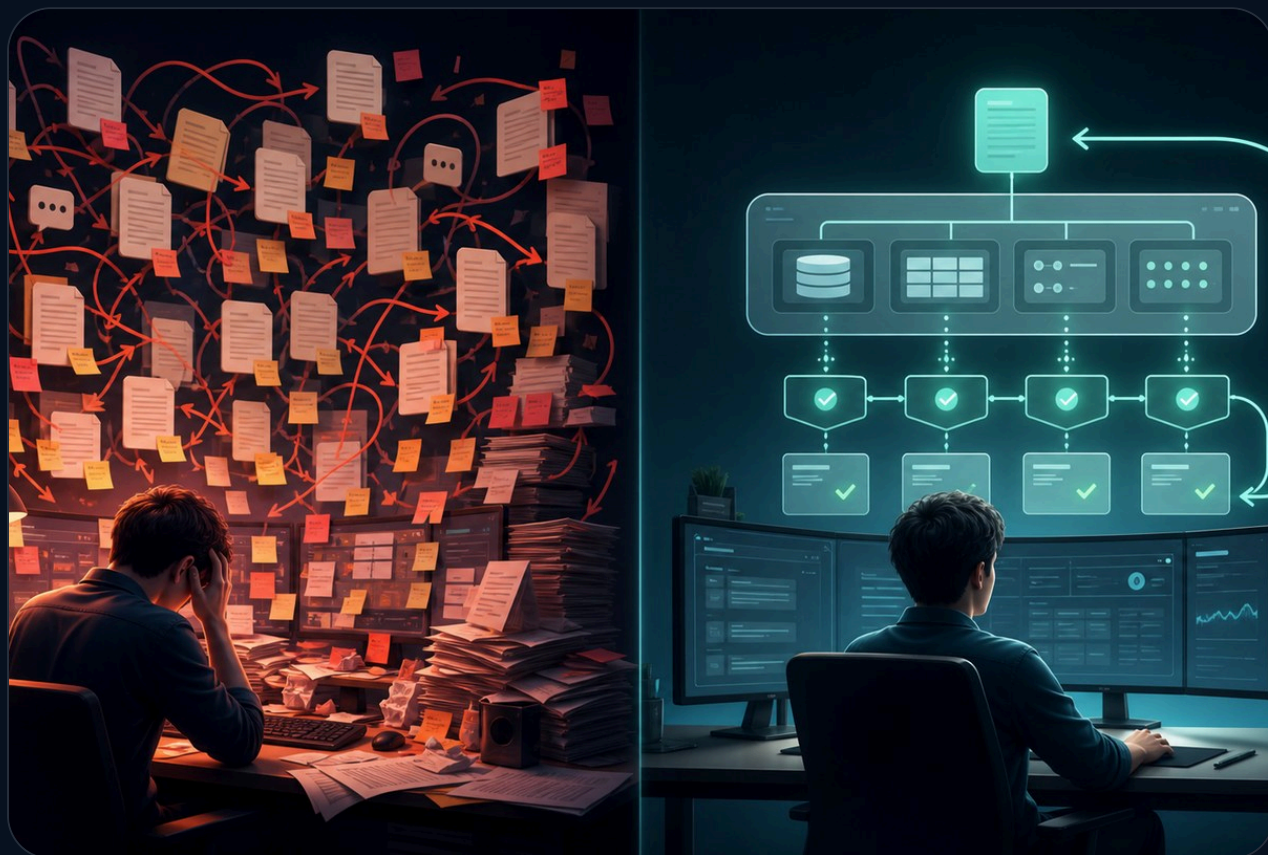
잠깐 멈춰서

**“올바른 바이트 코딩은
사용할수록 정리됩니다.”**

새 판단이 프롬프트 더미가 아니라
데이터 · 규칙 · 검증으로 남기 때문입니다.

비개발자도 느낄 수 있는 차이

작업의 흔적이 **짐**이 되다가, **자산**이 되다가



작업의 흔적이 이동해야 할 자리

프롬프트 조각

→ 공유된 업무 지식

파일 복사본

→ 단일 데이터 흐름

그때그때 예외

→ 명시된 업무 규칙

눈대중 확인

→ 검증 기준과 기록

다음번 더 복잡

→ 다음번 더 단순

정리되고 있다는 신호

YES PROMPT · 고개 끄덕이기

두 번째에는 이 네 가지가 줄어야 합니다

재설명

처음부터 맥락 말하기

수동 판단

사람이 매번 고르기

예외 패치

한 건만 억지로 고치기

불안한 확인

맞는지 눈으로 헤매기

줄었다면 구조가 생긴 것 · 늘었다면 아직 결과물 더미

한 사례에서 벗어나 보기

어떤 업무든 같은 방향으로 바뀝니다



서로 다른 업무, 같은 변환 공식

회의 녹음·메모 → 결정 · 담당자 · 기한 · 상태

메일·전화·채팅 → 유형 · 긴급도 · 담당 · 상태

아이디어·피드백 → 대상 · 메시지 · 검수 · 배포

결과 파일을 예쁘게 만드는 것이 아니라
다음 실행이 기억할 구조를 만듭니다.

비개발자는 원본을 주고, AGENT가 구조를 찾습니다

잘 설계된 프롬프트가 Agent에게 여섯 질문을 반복하게 합니다

① 원본

무엇이 들어오는가?

② 판단

사람이 매번 무엇을 고르는가?

③ 기억

다음에도 남길 칸은?

④ 상태

무엇이 어떤 조건에서 바뀌는가?

⑤ 검증

맞고 틀림을 어떻게 아는가?

⑥ 반복

새 사례를 어디에 넣는가?

모르는 것은 규칙으로 꾸미지 않습니다. '미정'으로 남기고, 사람이 답해야 할 질문으로 돌립니다.

그대로 복사해서 쓰는 마스터 프롬프트 ①

한 번 답하게 하지 말고 다음 단계까지 스스로 이끌게 합니다

80:20 → 20:80 업무 진화 Agent 프롬프트

너는 내 반복 업무를 비정형→정형→자동 실행으로 진화시키는 Agent야.
내가 업무 설명과 실제 원본을 주면 다음 순환을 스스로 주도해.

Explore: 원본 근거만 읽고 모호함·반복 판단·예외를 질문해.

Capture: 사람의 답을 확정/미정으로 구분해 기록해.

Structure: 데이터·상태·규칙·정상/경계/실패 검증으로 바꿔.

Automate: 수동·승인 후 Agent 실행·자동 실행 후보를 구분해.

Integrate: 연결된 도구가 있으면 승인 후 만들고 실제 결과를 검증해.

Learn: 새 예외를 항목·규칙·검증에 흡수하고 다시 실행해.

각 단계에서 산출물·사람이 결정할 것·다음 행동을 보여줘.

미정 정책과 위험한 실행에서는 질문하고 멈춰.

마스터 프롬프트 뒤에 붙이는 출력 계약 ②

Agent가 매 순환마다 구조와 자동화 수준을 갱신합니다

매 순환에서 유지할 산출물 계약

매 순환에서 아래 6개를 최신 상태로 유지해.

- ① 데이터: 이름·의미·형식·필수·허용값·원문 근거
- ② 상태: 시작·변경 조건·종료·예외 상태
- ③ 규칙: 조건·처리·결과·확정/미정
- ④ 검증: 정상·경계·실패 입력과 기대 결과
- ⑤ 재실행: 다음 사례를 넣는 최소 입력 양식
- ⑥ 실행 수준: 수동 / 사람 승인 후 Agent / 자동

다음 수준으로 올릴 때는 근거를 보여줘.

반복 규칙·검증 통과·도구 권한·실패 복구·기록이 없으면 올리지 마.

새 예외는 기존 구조의 어느 부분을 바꿨는지 표시하고 다시 검증해.

다음 작업 전에 기억을 회상하고, 끝난 뒤에는 배운 판단만 남깁니다

LLM Wiki 회상·학습 프롬프트

먼저 LLM Wiki 도구를 사용할 수 있는지 확인해.
없으면 기억했다고 말하지 말고 이번 대화에만 적용해.

[작업 시작 전]

이 프로젝트·산출물·스타일과 관련된 이전 결정·선호·
판단 기준·실패 원인을 좁게 회상해 적용해.
현재 요청과 충돌하면 현재 요청을 우선해.

[작업 종료 전]

작업이 끝나면 이번에 새로 확인된 내용 중
다음에도 재사용할 수 있는 암묵지만 후보로 분리해줘.
기존 관련 기억이 있으면 읽고 병합하고, 없을 때만 새로 저장해.
일회성 상태·민감정보·검증되지 않은 추측은 저장하지 마.

현재 요청 우선

관련 기억은 병합

재사용할 판단만

민감정보는 제외

마스터 프롬프트 검증 사례 · 행사 운영을 솔루션으로

처음 손에 있는 것은 흩어진 말과 파일입니다



카톡 원본

"저 처음 참가해요. AI 자동화에 관심 있고요. 친구와 같이 가도 될까요?"

담당자 메모

정원 30명 · 입금 후 확정 · 취소 시 대기자에게 연락

아직 모르는 것

동반 신청 인원 · 대표 연락처 · 자동 승급 여부

첫 질문: 어떤 앱을 만들까?

→ 다음에도 기억해야 할 것은 무엇일까?

AGENT가 스스로 선택한 EXPLORE 단계

첫 번째 출력은 앱이 아니라 좋은 질문입니다

AI가 묻습니다

- | 정원이 차면 어떻게 하나요?
- | 동반 신청은 몇 명까지인가요?

사람이 확정합니다

초과 신청은 접수 순서대로 대기

동반 신청은 최대 5명, 대표 연락처 필수

답을 모르면 '미정'으로 남깁니다. AI가 정책을 대신 만들게 두지 않습니다.

사람의 답을 받은 AGENT가 CAPTURE → STRUCTURE로 진행

말이 기억할 칸으로 바뀝니다

기억할 항목	형식	필수	허용값·조건
이름	짧은 글	예	빈 값 불가
연락처	전화번호	예	형식 검사
첫 참여	예/아니요	예	둘 중 하나
관심 주제	복수 선택	아니요	AI · 웹 · 데이터 · 기타
신청 단위	선택	예	개인 · 단체

AI가 초안을 만들고, **현업 담당자가 항목·형식·필수 여부를 확정**합니다.

구조를 읽은 AGENT가 상태·규칙으로 확장

말이 변화의 규칙으로 바뀝니다

신청



입금 확인



참가 확정



출석 / 취소

정원 초과이면 대기 · 취소 발생이면 대기 순서대로 승급 · 모든 변화는 기록

AGENT가 자동화 승격 전에 검증 기준을 생성

“잘 돼야 해요”를 확인 가능한 사례로 바꿉니다

정상

신청 20명

20명 모두 접수, 입금 확인 후 확정

경계

31번째 신청

정원 30명일 때 대기 1번

실패

잘못된 연락처

저장·발송 전에 오류 표시

세 사례의 기대 결과를 사람이 확인한 뒤, 다음 실행의 기준으로 남깁니다.

미니 데모 · 정형 구조를 재사용하고 새 예외만 반환

다음 달 행사를 넣어봅니다

9월 부산 모각코

정원	40명
신청	43명
확정 취소	2명
새 조건	단체 신청 있음

같은 규칙으로 처리

처리 결과

확정	40명
대기	1명
자동 승급	2명
검토 필요	단체 신청 규칙

사람이 새 정책을 답하면 AGENT가 LEARN 단계로 진행

한 번 판단하고 시스템의 자산으로 흡수합니다



예외를 재발하지 않는 구조로 바꾸는 네 단계

1 새 예외 발견
단체 신청은 기존 규칙으로 처리 불가



2 사람이 정책 결정
최대 5명 · 대표 연락처 필수



3 데이터·규칙·검증 추가
판단을 시스템이 다시 쓸 구조로 기록



4 다음부터 자동 처리
같은 예외를 다시 설명하지 않음

80:20 → 20:80은 한 번에 점프하지 않습니다

Agent가 현재 단계를 진단하고 다음 자동화 후보를 먼저 제안합니다

① 비정형 탐색

Agent가 질문·근거 수집

② 구조 초안

시트·체크리스트로 정리

③ 입력 정형화

폼·스키마로 빠짐 방지

④ Agent 실행

도구 사용 · 사람 승인

⑤ 자동 순환

검증·기록 · 예외는 반환

사람은 새 판단만 답합니다. 단계 진단·산출물 갱신·다음 제안·재검증은 Agent가 이어갑니다.

업무 데이터는 시트·DB에, 재사용할 판단은 LLM WIKI에

기억을 많이 쌓는 것이 아니라 관련 판단을 다음 실행에 이어줍니다



시작 전 회상 → 작업 → 확인 → 병합

1 관련 기억을 좁게 회상
프로젝트 · 산출물 · 스타일 기준



2 현재 요청과 비교해 적용
충돌하면 지금 요청이 우선



3 결과를 사람이 확인·교정
원하는 방향과 실패 원인을 확인



4 재사용할 판단만 병합
민감정보 · 일회성 상태 · 추측은 제외

다음 작업을 시작할 때 관련 기억을 다시 회상



실습 · 내 업무 하나를 가져오세요

4분 동안 내 업무를 넣고 Agent가 다음 단계를 이끌게 합니다

화면을 보며 그대로 입력하세요

너는 내 반복 업무를 80:20→20:80으로 진화시키는 Agent야.
아래 업무 설명과 개인정보를 가린 실제 원본만 근거로 시작해.

1. 원본·반복 판단·기억·상태·예외·검증을 표로 보여줘.
2. 추측하지 말고 미정 정책은 중요한 질문 3개로 물어봐.
3. 내가 답하면 구조를 갱신하고 다음 단계를 먼저 제안해.
체크리스트 → 시트 → 폼 → Agent 실행 → 자동화 중
지금 가능한 수준과, 다음 수준에 필요한 조건을 알려줘.

[내 업무 설명]

[개인정보를 가린 실제 원본 1건]



04:00

4분 타이머 시작

예: 회의록 · 신청 접수 · 보고서 · 콘텐츠 제작

자주 반복되고, 자주 틀리고, 판단 비용이 큰 것부터

①

반복 빈도

매주·매달 다시 하는가?

②

실수 비용

틀리면 고객·시간·돈에 영향?

③

설명 비용

새 담당자에게 오래 설명?

완료를 판정하는 법

“됐다”가 아니라 다섯 가지 변화를 측정합니다

✓ 재실행 시간이 줄었다

✓ 재설명 문장이 줄었다

✓ 수동 판단 수가 줄었다

✓ 오류를 먼저 발견한다

✓ 다른 사람도 실행한다

→ 두 번째가 더 쉽다

성숙의 순환

사람이 다음 명령을 만드는 단계에서 Agent가 다음 구조를 제안하는 단계로



내일부터 AGENT에게 맡길 세 질문

① 내 비정형 원본에서 무엇을 찾아야 하는가?

② 무엇을 구조·규칙·검증으로 옮길 수 있는가?

③ 다음 자동화 단계는 무엇이며, 무엇이 더 필요한가?

한 문장으로 마칩니다

**“Agent가 답만 만들게 하지 말고,
다음 정형화와 자동화를 스스로 이끌게 하십시오.”**

비정형에서 정형으로 · 수동에서 자동으로 · 80:20에서 20:80으로

Q&A · 발표 50분 이후 별도 6분

질문을 받겠습니다

자주 나오는 질문

- 처음부터 완벽하게 구조화해야 하나요?
- AI가 잘못 추출하면 어떻게 하나요?
- Agent가 정말 다음 단계를 알아서 이끄나요?

짧은 답

완벽하게가 아니라 반복 판단 하나씩 옮깁니다.

중요한 값은 사람의 확인과 검증 규칙을 둡니다.

운영 프롬프트와 도구가 있으면 Agent가 질문·구조화·실행·재검증을 이어갑니다.